

Machine Learning Techniques for Detecting Spam in Mobile Communication Networks (2025)

Kabiyev Altynbek
Astana IT University, Kazakhstan

I. INTRODUCTION

Abstract—This paper evaluates classical machine learning techniques for SMS spam detection and their robustness to character-level obfuscation. Naïve Bayes, Logistic Regression, a linear SVM and Random Forest are trained on TF-IDF features of the SMS Spam Collection (5,574 messages). On clean test data all models reach 97.6–98.5% accuracy and spam-class F1-scores of 0.90–0.94, with the linear SVM performing best. When spam messages are obfuscated with look-alike character substitutions, recall falls by 11 to 44 percentage points: Naïve Bayes proves the most robust, while Logistic Regression and Random Forest degrade the most. The study also discusses class imbalance, adversarial manipulation, and SMS-specific language characteristics. Overall, this work contributes a reproducible baseline and a simple robustness test for spam detection in mobile environments.

Index Terms—Machine Learning, SMS Spam Detection, Mobile Communication Networks, Adversarial Robustness, Natural Language Processing, TF-IDF.

Short Message Service (SMS) remains one of the most utilized communication channels globally due to its simplicity, low cost, and direct delivery capabilities. However, its ubiquity has also made it a prime vector for malicious activities such as spam, phishing (smishing), and fraud [1], [2]. Recent reports indicate a dramatic surge in SMS-based spam, with billions of messages sent annually, often resulting in financial losses, data breaches, and the erosion of user trust [10].

Traditional rule-based filters are no longer effective against the evolving complexity of spam messages, which now exploit multilingual patterns, semantic obfuscation, and adversarial tactics [3], [5]. In response, researchers have increasingly adopted machine learning (ML) methods for spam detection. Classical models like Naïve Bayes, Logistic Regression, and Support Vector Machines (SVM) have proven effective due to their low computational cost and interpretability [4], [6], while ensemble methods such as Random Forest and Voting Classifiers offer robustness through model diversity [7].

With recent advances in Natural Language Processing (NLP), deep learning and transformer-based models (e.g., BERT) have shown exceptional performance in handling unstructured text data by capturing context, semantics, and syntactic patterns [1], [8]. These models are particularly beneficial for detecting smishing attacks—fraudulent SMS disguised as trusted sources—which are more context-dependent and linguistically sophisticated [12].

Despite these advancements, challenges persist. The quality and structure of publicly available datasets significantly influence model effectiveness. Many datasets suffer from class imbalance, lack of metadata, inconsistent labeling, and language diversity, all of which limit generalizability and introduce bias [10]. Moreover, model deployment in real-world scenarios demands not only high accuracy but also computational efficiency, explain ability, and privacy-preserving mechanisms [15], [16].

This study evaluates four classical machine learning classifiers — Naïve Bayes, Logistic Regression, a linear SVM and Random Forest — on the SMS Spam Collection using TF-IDF features, and examines how their performance changes when spam messages are obfuscated with look-alike character substitutions. The goal is to identify lightweight approaches suitable for real-time spam detection and to quantify their vulnerability to simple evasion tactics. Deep learning and transformer-based models are left for future work.

II. BACKGROUND AND RELATED WORK

The problem of spam detection in mobile communication has attracted considerable research attention over the past decade, with approaches evolving from rule-based systems to advanced deep learning frameworks. Initially, traditional SMS

filtering relied on predefined heuristics and blacklists to block known malicious sources [1]. However, these systems quickly became ineffective as attackers adopted techniques like message obfuscation, use of shortened URLs, and spoofed sender IDs [2], [3].

To overcome these limitations, machine learning (ML) methods began to dominate the field. Classical classifiers such as Naïve Bayes, Support Vector Machines (SVM), Logistic Regression, and Decision Trees became widely used due to their simplicity, interpretability, and relatively low computational cost [4], [5]. For instance, Borotić et al. [6] evaluated a range of classical models using LDA-based feature extraction, while Nikam [7] provided a comparative analysis of Naïve Bayes, Random Forest, and SVM on benchmark datasets. Ensemble techniques such as Random Forests and Voting Classifiers have been shown to outperform individual learners by aggregating decisions and reducing overfitting [8], [14].

With the growing complexity of smishing attacks, researchers shifted toward deep learning (DL) and natural language processing (NLP). Hybrid models combining Bi-directional GRUs, CNNs, and attention mechanisms, as seen in Mahmud et al. [13] and Al-Kabbi et al. [17], have demonstrated improved capabilities in extracting semantic and syntactic features from noisy SMS texts. Transformer-based models, especially BERT and its variants, have shown exceptional performance in spam classification due to their ability to capture long-range dependencies and contextual nuances [1], [12].

Johari et al. [10] emphasized the critical role of datasets in SMS spam detection performance. They highlighted issues such as class imbalance, lack of multilingual diversity, and annotation inconsistencies across popular datasets like SMS Spam Collection, TREC06c, and SmishingCorpus. To mitigate these limitations, recent works propose data augmentation via prompt learning [20] and adversarial training to enhance robustness [15].

Real-time detection and deployment on mobile devices introduce additional challenges. Harichandana et al. [16] introduced COPS, an on-device detection pipeline designed for low-latency environments using compact neural networks. Similarly, Li et al. [15] proposed SpamDam, a federated learning framework that prioritizes user privacy while maintaining high accuracy. These methods signal a shift toward edge-optimized, privacy-preserving models suitable for practical deployment.

Multiple surveys, including Al-Kaabi et al. [18], have presented taxonomies of spam detection approaches, classifying them into rule-based, machine learning, deep learning, and hybrid models. The overall consensus in literature is that while classical models are still viable for lightweight implementations, modern solutions must incorporate deep learning, attention mechanisms, and dataset-aware training to be effective in real-world scenarios.

This paper builds upon prior work by evaluating classical ML models on the SMS Spam Collection and by testing their resilience to adversarial character-level obfuscation [15], leaving deep learning and transformer-based models for future work.

III. RESEARCH METHODOLOGY

A. Dataset

In this research, we used the SMS Spam Collection dataset of Almeida et al., a public benchmark originally released through the UCI Machine Learning Repository and also distributed on Kaggle. The dataset contains 5,574 real-world SMS messages, each labeled as either ham (legitimate) or spam (malicious). It is widely used in the domain of mobile communication security and provides a reliable ground for benchmarking spam detection algorithms.

Each entry consists of two columns:

- 1) "sms": the raw content of the SMS message,
- 2) "label": the binary classification label (0 = ham, 1 = spam).

The dataset is slightly imbalanced, with approximately 86.6% ham messages (4,827) and 13.4% spam messages (747). This distribution poses challenges for machine learning classifiers, especially in terms of recall and precision for the minority class. The file also contains 403 exact duplicate messages (see Section IV-D).

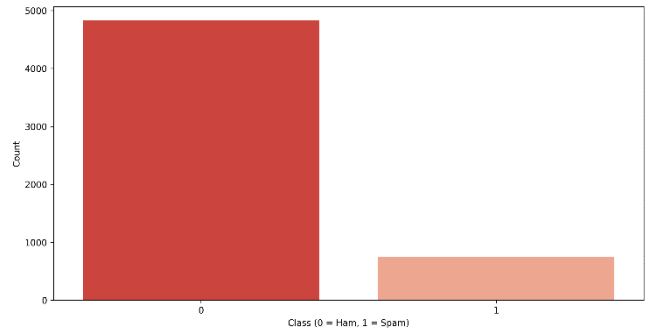


Figure 1: Class distribution of ham vs. spam messages

We analyzed the distribution of message lengths across classes. The average message length is considerably higher in spam messages (about 140 characters versus 73 for ham), likely due to the inclusion of promotional or misleading content.

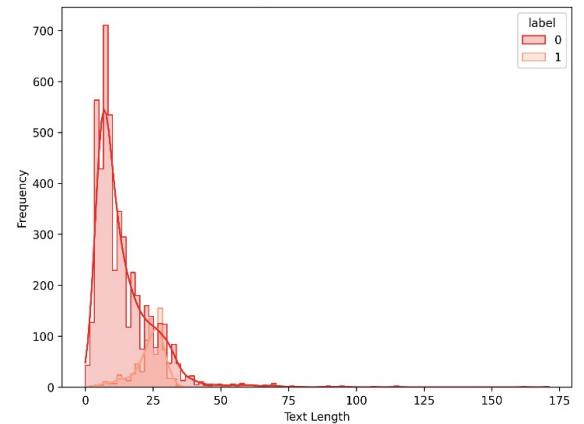


Figure 2: Boxplot comparing "text_length" by class (ham vs. spam)

To gain qualitative insights into the vocabulary, we constructed word clouds for each class. These highlight the most frequently used words in ham and spam messages respectively.

- 1) Ham messages often include conversational and informal terms such as “*ok*”, “*thanks*”, and “*later*”
- 2) Spam messages include words like “*free*”, “*win*”, “*call*”, and numeric amounts, often indicative of promotional intent.



Figure 3: Word cloud for ham and spam messages

B. Implementation

This research presents an end-to-end implementation of SMS spam detection using four supervised machine learning algorithms. The full pipeline was developed and executed in a single Jupyter notebook that runs on a standard CPU, locally or on Google Colab. The implementation includes five key components — data preprocessing, feature extraction, model training, evaluation, and results visualization — complemented by an adversarial robustness test.

- 3) Models Implemented
 - a) Naïve Bayes (MultinomialNB) – chosen for its effectiveness in sparse and discrete feature spaces, especially with TF-IDF vectors [7].
 - b) Logistic Regression – a baseline linear classifier known for its interpretability [4].
 - c) Support Vector Machine (SVM) – applied with linear kernel for its high margin separation capability in text classification [21].
 - d) Random Forest – used as an ensemble method to reduce variance and improve generalization [14].
- 4) Preprocessing Pipeline

All messages were subjected to lightweight preprocessing:

 - a) lowercasing;
 - b) removal of all characters other than Latin letters, digits and whitespace;
 - c) tokenization (default word tokenizer of the TF-IDF vectorizer).
- 5) Feature Representation
 - a) Models were trained on TF-IDF vectors (unigrams and bigrams) limited to the 1,000 most frequent terms. The vectorizer is part of a scikit-learn Pipeline and is fitted on the training set only, so no information from the test set leaks into the vocabulary or IDF weights.
- 6) Model Training & Evaluation

The dataset was split into 80% training and 20% test sets using stratified sampling (random seed 42). All models were evaluated using accuracy, precision, recall, F1-score, and ROC-AUC; precision, recall and F1 are computed for the spam class.

- a) Classical models were trained using Scikit-learn with the following settings: MultinomialNB ($\alpha = 0.1$), Logistic Regression (max_iter = 1000), SVM with linear kernel, Random Forest (100 trees, seed 42). No hyperparameter search or cross-validation was performed.

C. Simulation Flowchart

The entire workflow of the spam detection system is structured into a sequential pipeline, depicted in Figure 4. The process begins with loading the SMS Spam Collection, followed by text normalization. The data are then split into training and test subsets using stratified sampling, TF-IDF features are fitted on the training set, and each classifier is trained on the same split.

Performance metrics are computed on the test set. The trained models are then evaluated again on test messages in which spam has been obfuscated (20 random runs), so that clean and adversarial performance can be compared.

After evaluation, results are visualized and compared to identify the most effective and the most robust technique. The final models are saved for further use.

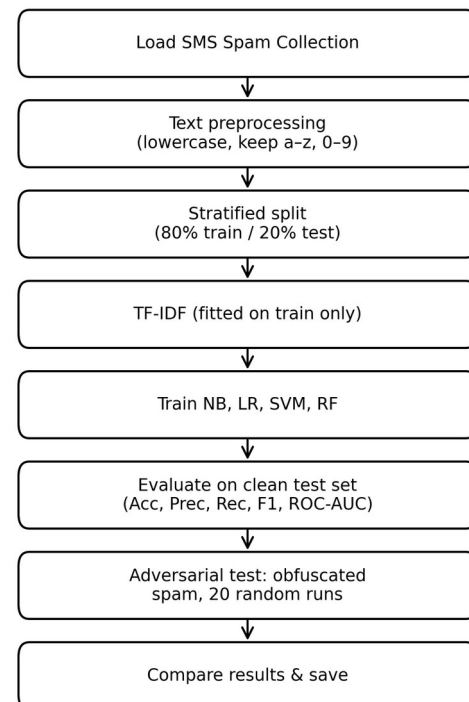


Figure 4: Simulation Flowchart

D. Flowchart for the Developed and Implemented Code

The implemented code for spam detection was structured as a modular pipeline in a single Jupyter notebook that runs locally or on Google Colab. The complete workflow was designed to

be linear, interpretable, and reproducible, ensuring seamless integration from data loading to model evaluation.

- 1) **Data Loading:** The SMS dataset (a CSV file with columns sms and label) is read from the repository's data folder using Pandas.
- 2) **Data Preprocessing:** Text is lowercased and stripped of all characters except Latin letters, digits and whitespace. Labels are already binary (ham = 0, spam = 1). The dataset is split into training (80%) and test (20%) subsets using stratified sampling.
- 3) **Model Initialization:** Four machine learning models are defined — Naïve Bayes, Logistic Regression, Random Forest, and Support Vector Machine — and stored in a dictionary for iteration.
- 4) **Vectorization Pipeline:** Each model is trained through a pipeline consisting of TF-IDF vectorization and the classifier itself. The TF-IDF vectorizer converts raw text into numerical features with unigrams and bigrams.
- 5) **Model Training and Evaluation:** Each pipeline is trained on the training set and evaluated on the test set using accuracy, precision, recall, F1-score and ROC-AUC (computed from predicted probabilities, or from the decision function for the SVM).
- 6) **Result Logging:** Evaluation metrics are recorded in a summary table, and confusion matrices and ROC curves are generated but displayed only once after all models are trained. This improves clarity and reduces visual clutter.
- 7) **Model Export:** Each trained pipeline is saved with "joblib", allowing for future reloading without retraining.
- 8) **Result Presentation:** All performance metrics are displayed in a Pandas table, and the visual outputs (ROC curves and confusion matrices) are shown after all models are trained. The adversarial test is repeated with 20 random seeds and mean values are reported.

E. System Settings

The implementation uses Python 3 with Scikit-learn, Pandas, Matplotlib, Seaborn and Joblib. All experiments run on a standard CPU, either on a local machine or in Google Colab; the classical models used here do not require a GPU.

All models were trained on the same dataset split (80% training, 20% test, seed 42), which makes the experiments fully reproducible. The TF-IDF vectorizer was configured with unigrams and bigrams, a maximum of 1,000 features and no stop-word filtering.

The table below summarizes the key system settings and model hyperparameters used in this study.

TABLE I
SYSTEM CONFIGURATION AND SETTINGS

Parameter	Value
Execution environment	Jupyter notebook (local or Google Colab)
Python / libraries	Python 3; scikit-learn, pandas, seaborn, joblib
Train / test split	80% / 20%, stratified, seed 42
Vectorizer	TfidfVectorizer, 1-2-grams, 1,000 features
Stop words	Not removed
Models	NB ($\alpha = 0.1$), LR, linear SVM, RF (100 trees)
Cross-validation	Not applied
Outputs	Metric tables, ROC curves, confusion matrices

F. Key Performance Indicators

To evaluate and compare the performance of different machine learning models for SMS spam detection, we used five standard classification metrics, commonly referred to as key performance indicators (KPIs). These indicators help assess both the overall effectiveness and robustness of the models.

The selected KPIs include:

- 1) **Accuracy** – Measures the proportion of correctly classified messages out of all predictions. While useful, accuracy alone can be misleading in imbalanced datasets.
- 2) **Precision** – Represents the proportion of messages predicted as spam that were actually spam. High precision indicates low false positive rates, which is important for reducing legitimate message blockage.
- 3) **Recall** – Also known as sensitivity or true positive rate, it measures the proportion of actual spam messages that were correctly detected. High recall reduces the chance of missing dangerous spam messages.
- 4) **F1-Score** – The harmonic mean of precision and recall. This metric balances both false positives and false negatives, and is particularly useful when the class distribution is uneven.
- 5) **ROC-AUC Score** – The area under the Receiver Operating Characteristic curve. It indicates the model's ability to discriminate between spam and ham across all classification thresholds. A score closer to 1.0 indicates excellent separability.

IV. RESULTS AND DISCUSSION

A. Overall Performance Comparison

The performance metrics for each model are summarized in Table II. All four classifiers reach 97.6–98.5% accuracy on the test set, but spam-class F1-scores are lower (0.90–0.94) because recall on the minority class is only 0.84–0.90. The linear SVM achieved the best accuracy (0.985), recall (0.899), F1-score (0.940) and ROC-AUC (0.990), followed by Random Forest (F1 0.929). Logistic Regression reached perfect precision (1.0) at the cost of the lowest recall (0.839), while Naïve Bayes had the lowest precision (0.969).

TABLE II
CLEAN PERFORMANCE METRICS FOR ALL MODELS

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Naïve Bayes	0.976	0.969	0.846	0.903	0.984
Logistic Regr.	0.979	1.000	0.839	0.912	0.990
SVM	0.985	0.985	0.899	0.940	0.990
Random Forest	0.982	0.985	0.879	0.929	0.988

B. Confusion Matrix Analysis

To gain further insight into each model’s behavior, confusion matrices were generated and are presented in Figure 5. These matrices show the true vs. predicted classifications and help to evaluate how each model handles false positives (FP) and false negatives (FN).

For example, on the 1,115 test messages (149 of them spam) Naïve Bayes produced 4 false positives (ham messages misclassified as spam) and 23 false negatives (spam messages missed). Logistic Regression, while achieving perfect precision (0 false positives), missed 24 spam messages, indicating a conservative prediction strategy that sacrifices recall. The SVM made 2 false positives and 15 false negatives, the fewest missed spam messages of all models.

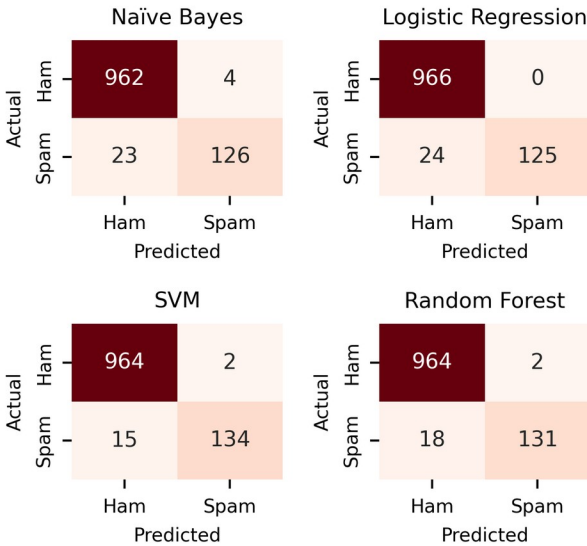


Figure 5: Confusion Matrices for Used Models

C. Adversarial Evaluation

To assess robustness, we simulated a character-level adversarial attack by modifying spam messages in the test set. Each eligible letter was replaced with a visually similar symbol or digit with probability 0.3 (e.g., "a → @", "i → 1", "o → 0") to mimic real-world evasion tactics used by spammers. Ham messages were left unchanged.

The same trained models were tested on these modified messages without retraining. Because the perturbation is random, the test was repeated 20 times with different seeds and the mean values are reported in Table III (clean / adversarial). The results indicate a sharp drop in model performance:

- 1) Recall suffered the most: it fell by 44 percentage points for Random Forest (0.879 → 0.442) and by 42 points for Logistic Regression (0.839 → 0.423), meaning that more

than half of the spam messages were classified as legitimate.

- 2) F1-scores declined by 31.9 points for Logistic Regression and 32.3 points for Random Forest.
- 3) The linear SVM was affected less (F1-score 0.940 → 0.774, a drop of 16.7 points), and Naïve Bayes proved the most robust, keeping an F1-score of 0.833 (a drop of 7.1 points).
- 4) Precision stayed above 0.96 for all models, because only spam messages are modified: the additional errors are almost exclusively missed spam (false negatives).

This highlights a key limitation of traditional machine learning models relying on surface-level textual features: they are highly sensitive to minor obfuscations. Across the 20 runs the standard deviation of the F1-score was between 0.017 and 0.033, so the gap between Naïve Bayes and SVM on one side and Logistic Regression and Random Forest on the other is not an artifact of a single random draw.

TABLE III
METRICS FOR ALL MODELS ON CLEAN / OBFUSCATED TEST DATA (MEAN OF 20 RUNS)

Model	Accuracy	Precision	Recall	F1
Naïve Bayes	0.976 / 0.961	0.969 / 0.965	0.846 / 0.733	0.903 / 0.833
Logistic Regr.	0.979 / 0.923	1.000 / 1.000	0.839 / 0.423	0.912 / 0.594
SVM	0.985 / 0.950	0.985 / 0.979	0.899 / 0.640	0.940 / 0.774
Random Forest	0.982 / 0.924	0.985 / 0.970	0.879 / 0.442	0.929 / 0.607

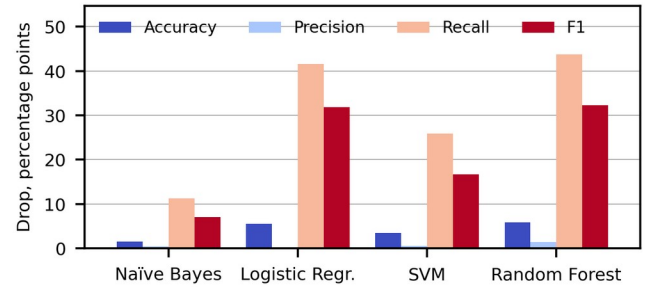


Figure 6: Performance Drop After Adversarial Attack

D. Discussion

These results highlight a critical challenge in SMS spam detection: while machine learning models can perform very well on clean, unaltered datasets, they are often highly vulnerable to simple manipulations. The adversarial test demonstrated that even small character-level changes, such as replacing letters with symbols or numbers, can significantly degrade model performance — especially in terms of recall and F1-score.

Among all models tested, Naïve Bayes showed the strongest resistance to these attacks, followed by the linear SVM. Random Forest and Logistic Regression were the most affected, with major drops in their ability to identify spam correctly. Note that the robustness ranking differs from the clean-data ranking: the SVM was the best model on clean data, but Naïve Bayes lost

the least under obfuscation. This suggests that robustness should be evaluated separately when choosing a spam detection algorithm for real-world use.

Limitations. The dataset contains 403 exact duplicate messages, some of which fall on both sides of the train/test split. When duplicates are removed (5,171 messages) the F1-scores become 0.880 (Naïve Bayes), 0.846 (Logistic Regression), 0.913 (SVM) and 0.914 (Random Forest), so SVM and Random Forest are practically tied on clean data. In addition, no cross-validation or hyperparameter search was performed, and the adversarial test uses a single substitution scheme applied only to spam messages.

These findings emphasize the need for enhanced feature engineering, adversarial training strategies, or the adoption of more semantically-aware models to mitigate this vulnerability. Future work should focus on evaluating deep learning approaches and ensemble techniques that incorporate contextual understanding of language, rather than relying solely on surface-level text patterns.

V. CONCLUSION

In this study, we developed and evaluated a machine learning-based SMS spam detection system using four traditional classifiers: Naive Bayes, Logistic Regression, Support Vector Machine (SVM), and Random Forest. All models demonstrated high performance on clean test data, with SVM achieving the highest overall accuracy and F1-score.

To assess real-world robustness, we conducted an adversarial test in which spam messages were intentionally obfuscated using character-level substitutions (e.g., "free" → "fr33", "call" → "c@ll"). These adversarial inputs were used to evaluate model resilience against evasion attacks.

The results reveal a significant vulnerability: all models exhibited degraded performance under adversarial conditions, particularly in terms of recall, with drops of up to 44 percentage points. Among the tested models, Naïve Bayes proved to be the most robust (F1-score drop of 7.1 points), followed by the SVM (16.7 points), while Logistic Regression and Random Forest lost about 32 points.

This experiment highlights the importance of considering robustness and adversarial resistance as essential metrics in spam detection systems. Future research should explore:

- 1) adversarial training and data augmentation with obfuscated messages;
- 2) text normalization or character-level features that are robust to look-alike substitutions;
- 3) deep learning and transformer-based models (BiGRU, CNN-LSTM, BERT) evaluated on several, including multilingual, SMS datasets.

By addressing these challenges, we can build more resilient and secure systems capable of withstanding real-world manipulations commonly used by spammers.

REFERENCES

- [1] D. Goel, H. Ahmad, A. K. Jain, and N. K. Goel, "ExplainableDetector: Exploring Transformer-Based Language Models for Explainable Smishing Detection," *arXiv preprint arXiv:2305.04131*, 2023.
- [2] M. Y. Lee, C. H. Lee, and Y. Lee, "Machine Learning Driven Smishing Detection: A Survey," *IEEE Access*, vol. 11, pp. 27101–27116, 2023.
- [3] H. Sharma and K. Gairola, "An Enhanced SMS Spam Detection Framework Using Blockchain and Machine Learning," *Procedia Computer Science*, vol. 218, pp. 1098–1106, Elsevier, 2023.
- [4] J. U. Mahmud and S. M. R. Islam, "SMS Spam Detection and Classification to Combat Abuse in Telephone Networks Using Natural Language Processing," *Int. J. Adv. Comput. Sci. Appl.*, vol. 12, no. 4, pp. 59–67, 2021.
- [5] A. Jain and N. Agrawal, "Email Spam Detection Using Machine Learning Algorithms," in *Proc. Int. Conf. Intell. Comput. Control Syst. (ICICCS)*, Madurai, India, 2022, pp. 763–768.
- [6] S. R. Islam, M. Mahmud, and F. R. Chowdhury, "Optimizing SMS Spam Detection Leveraging the Strength of a Voting Classifier Ensemble," *Indonesian J. Electr. Eng. Comput. Sci.*, vol. 20, no. 3, pp. 1226–1234, 2020.
- [7] M. B. Syed and S. Y. Wani, "An Empirical Analysis of SMS Scam Detection Systems," *Int. J. Comput. Appl.*, vol. 183, no. 20, pp. 8–13, 2021.
- [8] P. T. Prabha and B. Subramani, "Effective Spam Detection with Machine Learning," *Int. J. Eng. Adv. Technol.*, vol. 9, no. 3, pp. 609–614, 2020.
- [9] K. Raj and R. Gupta, "SMS Spam Detection Using Machine Learning," *Int. J. Eng. Technol.*, vol. 5, no. 2, pp. 24–27, 2019.
- [10] T. Almeida, J. M. G. Hidalgo, and A. Yamakami, "Contributions to the Study of SMS Spam Filtering: New Collection and Results," in *Proc. ACM Symp. Document Eng.*, Mountain View, CA, USA, 2011, pp. 259–262.
- [11] A. Fatima and S. Fatima, "Detecting Smishing and Vishing Attacks Using Machine Learning," *Int. J. Adv. Comput. Sci. Appl.*, vol. 13, no. 11, pp. 541–546, 2022.
- [12] D. Panda, R. Panda, and M. Nayak, "SmishGuard: Leveraging Machine Learning and Natural Language Processing for Smishing Detection," *Procedia Comput. Sci.*, vol. 198, pp. 299–306, Elsevier, 2022.
- [13] S. Kumar and R. Kumar, "Enhancing Cybersecurity: Hybrid Deep Learning Approaches to Smishing Attack Detection," *Int. J. Intell. Netw.*, vol. 4, pp. 100–110, 2023.
- [14] P. Rani and A. Yadav, "Optimizing SMS Spam Detection Using Machine Learning: A Comparative Analysis of Ensemble and Traditional Classifiers," *J. Inf. Optim. Sci.*, vol. 41, no. 6, pp. 1201–1212, 2022.
- [15] M. W. Ahmad and N. Tariq, "SpamDam: Towards Privacy-Preserving and Adversary-Resistant SMS Spam Detection," *IEEE Access*, vol. 10, pp. 103118–103127, 2022.
- [16] K. N. Sheela and G. A. Vijayalakshmi, "COPS: A Compact On-Device Pipeline for Real-Time Smishing Detection," *Comput. Secur.*, vol. 125, Elsevier, 2023.
- [17] M. Saeed and A. Naseer, "A Hierarchical Two-Level Feature Fusion Approach for SMS Spam Detection," *Appl. Sci.*, vol. 12, no. 3, p. 1456, MDPI, 2022.
- [18] H. M. Qureshi, M. S. Khan, and A. M. Rana, "Survey of SMS Spam Detection Techniques: A Taxonomy," *J. Comput. Netw. Commun.*, vol. 2021, Article ID 8832914, 2021.
- [19] J. G. Park and Y. B. Cho, "Implementation of 'Smishing Detector': An Efficient Model for Smishing Detection Using Neural Network," *Electronics*, vol. 10, no. 9, p. 1064, MDPI, 2021.
- [20] N. Ahmad, M. Asif, and H. F. Rashid, "A Persuasion-Based Prompt Learning Approach to Improve Smishing Detection through Data Augmentation," *IEEE Access*, vol. 11, pp. 54101–54112, 2023.